

Sprint 4 Structured Notes: Bash Scripting Basics

1. Current Progress

You are now in **Sprint 4** of the Relational Databases learning path.

The full learning path is:

Tools → Terminal → Bash → Bash Scripting → Git → PostgreSQL → SQL → Normalization, Security, Performance

So far, you have completed the foundation stages:

Sprint 1: Tools and VS Code

In Sprint 1, you learned how to prepare your coding environment.

You learned about:

- VS Code
- Code editors and IDEs
- Workspaces
- Project folders
- `index.html`
- Live Server
- Basic VS Code shortcuts
- Beginner extensions

Simple meaning:

Sprint 1 helped you prepare the place where you will write and manage project files.

Sprint 2: Terminal, Shell, and Command Line Basics

In Sprint 2, you learned what happens when you type commands.

You learned about:

- Terminal
- Command line
- Shell
- Bash
- Command history
- Auto-complete
- Clearing the terminal screen

- Stopping a running command

Simple meaning:

Sprint 2 helped you understand the environment where commands are typed and run.

Sprint 3: Bash File and Folder Commands

In Sprint 3, you learned how to use Bash commands to work with files and folders.

You learned commands such as:

- pwd
- cd
- ls
- mkdir
- touch
- echo
- cat
- less
- mv
- cp
- rm
- >
- >>

Simple meaning:

Sprint 3 taught you how to move around folders, create files, read files, rename files, copy files, delete files, and write text into files using commands.

Sprint 4: Bash Scripting Basics

Now you are learning how to save Bash commands inside a file and run them together.

In Sprint 3, you typed commands one by one.

In Sprint 4, you put commands into a script file.

Simple meaning:

Sprint 4 teaches you how to turn Bash commands into a reusable script.

2. Main Goal of Sprint 4

The main goal of Sprint 4 is:

Turn Bash commands into a small reusable script.

In Sprint 3, you typed commands manually.

Example:

```
mkdir test-folder  
touch readme.md  
echo "Hello" > readme.md  
cat readme.md
```

In Sprint 4, you learn how to place commands like these inside one file.

Then you can run that file whenever you want.

Simple meaning:

A Bash script is a file that contains Bash commands.

3. Why Sprint 4 Is Important

Sprint 4 is important because developers often repeat the same commands many times.

Instead of typing the same commands again and again, you can save them inside a script.

Later, Bash scripting can help you with:

- Creating project folders automatically
- Running repeated setup commands
- Backing up files
- Running developer tools
- Running Git commands
- Preparing SQL project folders
- Calling other programs from the terminal
- Automating small tasks

Simple meaning:

Bash scripting saves time because it lets the computer run repeated commands for you.

4. What Is a Bash Script?

A **Bash script** is a normal text file that contains Bash commands.

Instead of typing commands one by one in the terminal, you write them in a file.

Example script:

```
#!/bin/bash  
  
echo "Hello"  
echo "I am learning Bash scripting"
```

When you run this file, Bash runs the commands from top to bottom.

Simple meaning:

A Bash script is a saved list of terminal commands.

5. Bash Script File

A Bash script is usually saved as a file.

Many Bash script files use the `.sh` extension.

Example:

```
hello.sh
```

The `.sh` extension means it is a shell script file.

Simple meaning:

A script file is a file where you write commands that Bash can run.

Important note:

The `.sh` extension is helpful because it tells people the file is a shell script, but the content inside the file is what really matters.

6. Why Do We Use Bash Scripts?

Bash scripts are useful when you want to repeat the same task many times.

Imagine you need to run these commands every day:

```
mkdir backup  
cp notes.md backup/  
echo "Backup completed"
```

Typing them manually every day is slow and boring.

So you can put them into a script:

```
#!/bin/bash  
  
mkdir backup  
cp notes.md backup/  
echo "Backup completed"
```

Then you can run the script whenever you need it.

Simple meaning:

Bash scripts save time and reduce repeated work.

7. What Is a Shebang?

The first line of many Bash scripts is:

```
#!/bin/bash
```

This line is called a **shebang**.

The shebang tells the computer which program should run the script.

Here:

```
#!/bin/bash
```

means:

Run this script using Bash.

Simple meaning:

The shebang tells the system which interpreter to use.

8. Why the Shebang Is Important

A computer may have different shells or interpreters.

Examples:

- Bash
- Sh
- Zsh
- PowerShell

The shebang helps the system know which one should run the script.

Example:

```
#!/bin/bash
```

This tells the system:

Use Bash to run this file.

Simple meaning:

The shebang gives the script a clear running instruction.

9. What Is an Interpreter?

An **interpreter** is a program that reads and runs code.

For Bash scripts, the interpreter is usually Bash.

Example:

```
#!/bin/bash
```

This means the Bash interpreter is used to run the script.

Simple meaning:

An interpreter reads your script and runs the commands inside it.

10. Common Bash Interpreter Paths

Common interpreter paths are:

```
/usr/bin/bash  
/bin/bash  
/bin/sh
```

```
/bin/bash
```

This is a common path for Bash.

Example:

```
#!/bin/bash
```

Simple meaning:

Use Bash from `/bin/bash` .

```
/usr/bin/bash
```

This is another possible Bash location.

Example:

```
#!/usr/bin/bash
```

Simple meaning:

Use Bash from `/usr/bin/bash` .

```
/bin/sh
```

This uses `sh`, which is another shell.

Example:

```
#!/bin/sh
```

Simple meaning:

Use `sh` to run the script.

Beginner note:

For this sprint, use:

```
#!/bin/bash
```

11. Creating Your First Bash Script

Step 1: Create a script file

Use the `touch` command:

```
touch hello.sh
```

Meaning:

Create a file called `hello.sh`.

Step 2: Open the file

Open `hello.sh` in VS Code or another editor.

Step 3: Add script content

Write this inside the file:

```
#!/bin/bash
```



```
echo "Hello"  
echo "I am learning Sprint 4"
```

Step 4: Save the file

Save the file before running it.

Step 5: Run the script

Run:

```
bash hello.sh
```

Expected output:

```
Hello  
I am learning Sprint 4
```

Simple meaning:

You created a script file and asked Bash to run it.

12. How Bash Runs a Script

When you run:

```
bash hello.sh
```

This is what happens:

1. Bash opens the file `hello.sh`.
2. Bash reads the commands inside the file.
3. Bash runs the commands from top to bottom.
4. The terminal shows the output.

Simple meaning:

Bash reads the script line by line and runs each command.

13. What Is `echo` in a Script?

You already used `echo` in Sprint 3.

In a script, `echo` is used to print messages in the terminal.

Example:

```
echo "Starting backup..."  
echo "Backup finished."
```

Output:

```
Starting backup...  
Backup finished.
```

Simple meaning:

`echo` shows messages in the terminal.

14. Why `echo` Is Useful in Scripts

`echo` helps you understand what the script is doing.

Example:

```
echo "Creating backup folder"  
mkdir backup  
echo "Backup folder created"
```

The messages help the user follow the script.

Simple meaning:

`echo` is used for logging messages while the script is running.

15. What Is a Variable?

A variable is a name that stores a value.

Example:

```
name="Maleesha"  
echo "Hello $name"
```

Output:

```
Hello Maleesha
```

Here:

```
name="Maleesha"
```

stores the value `Maleesha`.

Then:

```
$name
```

uses that value.

Simple meaning:

A variable stores information that you can use later.

16. Bash Variable Rule: No Spaces Around `=`

In Bash, do not put spaces around `=` when creating a variable.

Correct:

```
name="Maleesha"
```

Wrong:

```
name = "Maleesha"
```

Simple meaning:

In Bash variables, write the name, equal sign, and value together with no spaces.

17. What Is Variable Interpolation?

Variable interpolation means using the value of a variable inside a command or text.

Example:

```
server="prod"  
echo "Pulling $server"
```

Output:

```
Pulling prod
```

Here, `$server` is replaced by the value `prod`.

Simple meaning:

Variable interpolation means Bash replaces the variable name with its value.

18. Why We Use `$` Before Variable Names

In Bash, when you create a variable, you do not use `$`.

Example:

```
server="prod"
```

But when you use the value of the variable, you use `$`.

Example:

```
echo "$server"
```

Simple meaning:

No `$` when creating the variable. Use `$` when reading the variable value.

19. What Is an Array?

An array is like a list.

Example:

```
servers=( "prod" "dev" )
```

This array contains two values:

```
prod  
dev
```

Simple meaning:

An array stores multiple values in one variable.

20. Understanding the `servers` Array

Example:

```
servers=( "prod" "dev" )
```

Here:

- `servers` is the array name.
- `prod` is one value.
- `dev` is another value.

In the example:

- `prod` means production site.
- `dev` means development site.

Simple meaning:

The array keeps both server names together in one list.

21. What Is `prod`?

`prod` is a short form of **production**.

A production site or production server is the real live system.

Example:

If a website is already used by real users, that system is usually called production.

Simple meaning:

`prod` means the real live version.

22. What Is `dev`?

`dev` is a short form of **development**.

A development site or development server is used for testing and building before changes go live.

Simple meaning:

`dev` means the testing or development version.

23. What Is a `for` Loop?

A `for` loop repeats commands for each item in a list or array.

Example:

```
servers=( "prod" "dev" )  
  
for server in "${servers[@]}"  
do  
    echo "Pulling $server"  
done
```

Output:

```
Pulling prod
Pulling dev
```

Simple meaning:

A `for` loop repeats commands for every item in a list.

24. Basic Structure of a `for` Loop

A Bash `for` loop usually looks like this:

```
for item in "${list[@]}"
do
    command
    command
done
```

Meaning:

- `for` starts the loop.
- `item` holds one value at a time.
- `in` tells Bash where the values come from.
- `"${list[@]}"` means all values in the list.
- `do` starts the loop body.
- The commands inside run for each item.
- `done` ends the loop.

Simple meaning:

A loop repeats the same commands for each value.

25. Understanding the Loop Step by Step

Script:

```
servers=("prod" "dev")

for server in "${servers[@]}"
do
```

```
    echo "Pulling $server"  
done
```

Step 1: Create the array

```
servers=("prod" "dev")
```

This creates a list with two values.

Step 2: Start the loop

```
for server in "${servers[@]}"
```

This means:

Take each value from the `servers` list one by one.

Step 3: First loop round

First value:

```
server="prod"
```

Command runs:

```
echo "Pulling $server"
```

Output:

```
Pulling prod
```

Step 4: Second loop round

Second value:

```
server="dev"
```

Command runs:


```
echo "Pulling $server"
```

Output:

```
Pulling dev
```

Step 5: End the loop

```
done
```

This ends the loop.

Simple meaning:

The loop runs once for `prod` and once for `dev`.

26. What Does `do` Mean?

In a Bash loop, `do` starts the block of commands that should repeat.

Example:

```
for server in "${servers[@]}"
do
    echo "Pulling $server"
done
```

Here, the command after `do` runs for each server.

Simple meaning:

`do` starts the repeated part of the loop.

27. What Does `done` Mean?

In a Bash loop, `done` ends the loop block.

Example:

```
for server in "${servers[@]}"
do
    echo "Pulling $server"
done
```

Simple meaning:

`done` tells Bash that the loop is finished.

28. What Does "\${servers[@]}" Mean?

This part:

```
"${servers[@]}"
```

means:

Use all items inside the `servers` array.

If the array is:

```
servers=( "prod" "dev" )
```

Then:

```
"${servers[@]}"
```

represents:

```
prod
dev
```

Simple meaning:

`${servers[@]}` means all values inside the array.

29. Why We Use Quotes Around `"${servers[@]}"`

The example writes:

```
"${servers[@]}"
```

The double quotes help Bash handle each array item safely.

For beginners, remember this pattern:

```
for server in "${servers[@]}"
```

Simple meaning:

Use quotes around array expansion to keep the values safe and separate.

30. What Is `rsync`?

`rsync` is a command used to copy or synchronize files.

It is often used for backups.

Example idea:

```
rsync --archive --verbose source destination
```

Simple meaning:

`rsync` **copies or syncs files from one place to another.**

31. Understanding the `rsync` Command

Example:

```
rsync --archive --verbose source destination
```

`rsync`

This is the command.

Meaning:

Copy or synchronize files.

`--archive`

This option keeps file details such as structure and permissions.

Simple meaning:

Copy in a careful backup-friendly way.

`--verbose`

This option shows more details while the command runs.

Simple meaning:

Show what is happening.

`source`

This is where the file comes from.

Simple meaning:

The starting location.

`destination`

This is where the file goes.

Simple meaning:

The ending location.

32. Full Curriculum Example

The Sprint 4 example has this structure:

```
#!/bin/bash

servers=("prod" "dev")

for server in "${servers[@]}"
do
    echo "Pulling $server"
    rsync --archive --verbose $server:/etc/nginx/conf.d/server.conf configs/
    $server.conf
done
```

This is a more realistic Bash script.

It uses:

- A shebang
- An array
- A `for` loop
- `do`
- `done`
- `echo`
- Variable interpolation
- `rsync`

33. Full Example Explained in Simple Words

Line 1

```
#!/bin/bash
```

This tells the system to run the script using Bash.

Line 2

```
servers=("prod" "dev")
```

This creates an array with two server names:

- `prod`
- `dev`

Line 3

```
for server in "${servers[@]}"
```

This starts a loop.

It takes each server name one by one.

Line 4

```
do
```

This starts the commands that will repeat.

Line 5

```
echo "Pulling $server"
```

This prints a message.

First it prints:

```
Pulling prod
```

Then it prints:

```
Pulling dev
```

Line 6

```
rsync --archive --verbose $server:/etc/nginx/conf.d/server.conf configs/  
$server.conf
```

This copies a file from the current server to a local folder.

Line 7

```
done
```

This ends the loop.

Simple meaning:

The script backs up config files from production and development servers.

34. Understanding This Part: `$server:/etc/nginx/conf.d/server.conf`

In the example:

```
$server:/etc/nginx/conf.d/server.conf
```

`$server` changes depending on the loop.

First loop:

```
prod:/etc/nginx/conf.d/server.conf
```

Second loop:

```
dev:/etc/nginx/conf.d/server.conf
```

Simple meaning:

The script uses the current server name to build the source path.

35. Understanding This Part: `configs/$server.conf`

In the example:

```
configs/$server.conf
```

`$server` changes depending on the loop.

First loop:

```
configs/prod.conf
```

Second loop:

```
configs/dev.conf
```

Simple meaning:

The script saves each server config using that server's name.

36. What Is NGINX in the Example?

The example path is:

```
/etc/nginx/conf.d/server.conf
```

This points to an NGINX configuration file.

For this sprint, you do not need to learn NGINX deeply.

You only need to understand that the script is copying a configuration file.

Simple meaning:

NGINX is just part of the example file path. The main lesson is how the script loops and copies files.

37. What Is a Binary?

A binary is a compiled program that the computer can run.

Examples of binaries or command-line programs:

```
bash  
ls  
cp  
rsync  
git  
psql
```


When your Bash script runs a command like:

```
rsync
```

it is calling another program.

Simple meaning:

A binary is a runnable program on your computer.

38. Bash Scripts Can Call Other Programs

A Bash script is not limited to only Bash built-in commands.

It can call other programs too.

Examples:

```
git status  
psql -U username -d database_name  
rsync --archive --verbose source destination
```

Simple meaning:

Bash scripts can run other command-line tools.

39. Advantages of Bash Scripting

Bash scripting has several advantages.

Advantage 1: Bash is available in many Unix environments

Bash is common in:

- Linux
- macOS
- Git Bash on Windows
- Servers

Simple meaning:

Bash scripts can run in many developer environments.

Advantage 2: No need to configure Node.js or Python for small tasks

For simple automation, Bash can be enough.

Simple meaning:

You can use Bash for small tasks without setting up a bigger programming environment.

Advantage 3: Bash scripts can call binaries

Scripts can run other programs like:

- rsync
- git
- psql
- ls
- cp

Simple meaning:

Bash scripts can control many command-line tools.

Advantage 4: You can test small parts directly in the terminal

Before writing a command in a script, you can test it in the terminal.

Example:

```
echo "Pulling prod"
```

If it works, you can add it to the script.

Simple meaning:

Test one small command first, then put it into the script.

40. Safe Beginner Habit: Test One Line First

Before putting a command into a script, test it directly in the terminal.

Example:

First test:

```
echo "Preparing folder: frontend"
```

Then add it to your script:

```
folders=( "frontend" "backend" )

for folder in "${folders[@]}"
do
    echo "Preparing folder: $folder"
done
```

Simple meaning:

Do not write a big script at once. Test small commands first.

41. Sprint 4 Practice Task

Your practice task is:

Create a script that loops through two folder names and prints a message for each one.

This task helps you practice:

- Creating a script file
 - Writing a shebang
 - Creating an array
 - Writing a `for` loop
 - Using `do` and `done`
 - Using `echo`
 - Using variable interpolation
-

42. Practice Task Step-by-Step

Step 1: Create a script file

```
touch sprint4.sh
```

Meaning:

Create a file called `sprint4.sh`.

Step 2: Open the file

Open `sprint4.sh` in VS Code.

Step 3: Add this script

```
#!/bin/bash

folders=("frontend" "backend")

for folder in "${folders[@]}"
do
    echo "Preparing folder: $folder"
done
```

Step 4: Run the script

```
bash sprint4.sh
```

Expected output:

```
Preparing folder: frontend
Preparing folder: backend
```

Simple meaning:

The script loops through two folder names and prints a message for each one.

43. Practice Script Explained

Script:

```
#!/bin/bash

folders=("frontend" "backend")

for folder in "${folders[@]}"
do
```

```
    echo "Preparing folder: $folder"  
done
```

```
#!/bin/bash
```

Run this script using Bash.

```
folders=("frontend" "backend")
```

Create an array with two folder names.

```
for folder in "${folders[@]}"
```

Take each folder name one by one.

```
do
```

Start the repeated commands.

```
    echo "Preparing folder: $folder"
```

Print the current folder name.

```
done
```

End the loop.

Simple meaning:

The script repeats the echo command for each folder in the array.

44. Sprint 4 Checkpoint

Your checkpoint is:

Write a script that loops through two folder names and prints a message for each one.

Your script should:

1. Have a shebang.
2. Create an array with two names.
3. Use a `for` loop.
4. Use `do` and `done`.
5. Print a message using `echo`.

6. Use a variable inside the message.

Checkpoint example:

```
#!/bin/bash

folders=("frontend" "backend")

for folder in "${folders[@]}"
do
    echo "Checking folder: $folder"
done
```

Expected output:

```
Checking folder: frontend
Checking folder: backend
```

Simple meaning:

This checkpoint proves that you can write a basic Bash loop inside a script.

45. Common Beginner Mistakes

Mistake 1: Adding spaces around `=`

Wrong:

```
name = "Maleesha"
```

Correct:

```
name="Maleesha"
```

Simple meaning:

Do not put spaces around `=` **when creating Bash variables.**

Mistake 2: Forgetting `done`

Wrong:

```
for folder in "${folders[@]}"
do
    echo "$folder"
```

Correct:

```
for folder in "${folders[@]}"
do
    echo "$folder"
done
```

Simple meaning:

Every `for` loop needs `done` at the end.

Mistake 3: Forgetting `$` before a variable name

Wrong:

```
echo "Checking folder"
```

This does not show the folder value.

Correct:

```
echo "Checking $folder"
```

Simple meaning:

Use `$folder` when you want to show the value stored in the variable.

Mistake 4: Confusing array name and loop variable

Example:

```
folders=("frontend" "backend")  
  
for folder in "${folders[@]}"
```

Here:

- `folders` is the full list.
- `folder` is one item at a time.

Simple meaning:

The array stores all items. The loop variable stores one item during each loop round.

Mistake 5: Forgetting to save the file before running

Problem:

You write the script but forget to save it.

Then the terminal may run the old version of the file.

Better habit:

Save the file first.

Then run:

```
bash sprint4.sh
```

Simple meaning:

Save first, run second.

Mistake 6: Running the wrong file name

Problem:

Your file is called:

```
sprint4.sh
```


But you run:

```
bash sprint.sh
```

That will not run the correct file.

Better:

```
bash sprint4.sh
```

Simple meaning:

Make sure the file name in the command matches the real script file name.

Mistake 7: Forgetting quotes around text

Wrong:

```
echo Hello Sprint 4
```

This may still work for simple text, but it is safer and clearer to write:

```
echo "Hello Sprint 4"
```

Simple meaning:

Use quotes around text messages to make them clear.

46. Sprint 4 Command and Syntax Summary

Command / Syntax	Simple Meaning
<code>touch hello.sh</code>	Create a script file
<code>#!/bin/bash</code>	Shebang: run script using Bash
<code>bash hello.sh</code>	Run the script with Bash
<code>echo "Hello"</code>	Print text in the terminal
<code>name="Maleesha"</code>	Create a variable

Command / Syntax	Simple Meaning
<code>\$name</code>	Use the variable value
<code>servers=("prod" "dev")</code>	Create an array/list
<code>"\${servers[@]}"</code>	Use all values in the array
<code>for server in "\${servers[@]}"</code>	Start a loop through the array
<code>do</code>	Start the loop body
<code>done</code>	End the loop body
<code>rsync --archive --verbose source destination</code>	Copy/sync files with details

47. Key Terms Summary

Term	Simple Meaning
Bash	A common shell used to run commands
Bash script	A file that contains Bash commands
Script file	A file that stores commands to run together
<code>.sh</code> file	A common file extension for shell scripts
Shebang	First line that tells the system which interpreter to use
Interpreter	Program that reads and runs code
<code>/bin/bash</code>	Common path to the Bash interpreter
<code>/usr/bin/bash</code>	Another possible Bash interpreter path
<code>/bin/sh</code>	Path to the <code>sh</code> shell
<code>echo</code>	Command used to print text
Variable	A name that stores a value
Variable interpolation	Replacing a variable name with its value
Array	A list of multiple values stored in one variable
<code>for</code> loop	A structure that repeats commands for each item
<code>do</code>	Starts the loop block
<code>done</code>	Ends the loop block
<code>prod</code>	Production / real live system

Term	Simple Meaning
dev	Development / testing system
rsync	Tool used to copy or synchronize files
Binary	A runnable program on your computer

48. Final Sprint 4 Understanding

After Sprint 4, you should understand that Bash commands do not always need to be typed one by one.

You can write them inside a script file and run them together.

You should now understand:

- What a Bash script is
- Why Bash scripts are useful
- What a shebang is
- What an interpreter is
- Common interpreter paths
- How to create a `.sh` script file
- How to run a script using `bash script_name.sh`
- How to use `echo` for messages
- How to create variables
- How to use variables with `$`
- What variable interpolation means
- How to create an array
- How to loop through an array
- What `do` and `done` mean
- What `"${array[@]}"` means
- What `rsync` does
- What a binary is
- Why Bash scripting is useful for automation

Simple final meaning:

Sprint 4 teaches you how to save repeated Bash commands inside a file and run them as a script. This is the foundation for automation, Git workflows, PostgreSQL project setup, and real developer tasks.

49. Sprint 4 Mini Assessment

Answer these questions before moving to Sprint 5.

1. What is a Bash script?
2. Why do we use Bash scripts?
3. What is a script file?
4. What does `.sh` usually mean?
5. What is a shebang?
6. What does `#!/bin/bash` mean?
7. What is an interpreter?
8. Name three common interpreter paths.
9. How do you create a script file called `hello.sh`?
10. How do you run `hello.sh` using Bash?
11. What does `echo` do?
12. Why is `echo` useful in scripts?
13. What is a variable?
14. What is the correct way to create a Bash variable?
15. Why is `name = "Maleesha"` wrong in Bash?
16. What does `$name` mean?
17. What is variable interpolation?
18. What is an array?
19. What values are inside this array: `servers=("prod" "dev")`?
20. What does `prod` mean?
21. What does `dev` mean?
22. What does a `for` loop do?
23. What does `do` mean in a loop?
24. What does `done` mean in a loop?
25. What does `"${servers[@]}"` mean?
26. What does `rsync` do?
27. What does `--archive` mean in the `rsync` example?
28. What does `--verbose` mean in the `rsync` example?
29. What is a binary?
30. Why is Bash scripting useful for developers?

50. Sprint 4 Completion Checklist

Tick these when completed:

- ☐ I can explain what a Bash script is.
- ☐ I can explain why Bash scripts are useful.
- ☐ I can create a script file using `touch`.
- ☐ I can write a shebang line.
- ☐ I understand what `#!/bin/bash` means.
- ☐ I can explain what an interpreter is.

- [] I know common interpreter paths like `/bin/bash`, `/usr/bin/bash`, and `/bin/sh`.
 - [] I can run a script using `bash script_name.sh`.
 - [] I can use `echo` inside a script.
 - [] I can create a variable without spaces around `=`.
 - [] I can use `$variable_name` to read a variable value.
 - [] I can explain variable interpolation.
 - [] I can create an array with two values.
 - [] I can write a `for` loop.
 - [] I can use `do` and `done` correctly.
 - [] I can explain what `"${servers[@]}"` means.
 - [] I can explain what `rsync` does in simple words.
 - [] I can explain what a binary is.
 - [] I completed the Sprint 4 checkpoint script.
-

51. Sprint 4 Checkpoint Task to Submit

Create a file called:

```
sprint4.sh
```

Write this script:

```
#!/bin/bash

folders=("frontend" "backend")

for folder in "${folders[@]}"
do
    echo "Checking folder: $folder"
done
```

Run it:

```
bash sprint4.sh
```

Expected output:

```
Checking folder: frontend
Checking folder: backend
```

Submit:

1. Your script code.
 2. Your terminal output.
 3. A short explanation of what the loop does.
-

52. End of Sprint 4 Notes

You are ready to move to Sprint 5 only after you can write and run the checkpoint script without copying blindly.

Before moving forward, make sure you can explain this script in simple words:

```
#!/bin/bash

folders=("frontend" "backend")

for folder in "${folders[@]}"
do
    echo "Checking folder: $folder"
done
```

Final simple meaning:

Sprint 4 teaches you how to turn simple Bash commands into a reusable script. This is the first step toward automation.